অধ্যায় ৬

Arrays: Data Collection, Array Methods এবং Practical Data Processing

শেখার লক্ষ্য

এই অধ্যায় শেষে তুমি Array তৈরি ও পরিবর্তন, Indexing, Traversal, `map` / `filter` / `reduce` / `find` / `some` / `every` / `sort` ব্যবহার, Reference ও Mutation-এর পার্থক্য, Shallow Copy, Nested Arrays, Real-World Inventory Data Processing এবং Automated Test লিখতে পারবে।

৬.১ Array কেন দরকার?

ধরো একটি দোকানে ৫০০টি পণ্যের নাম আছে। `product1`, `product2` এভাবে ৫০০টি Variable তৈরি করা বাস্তবসম্মত নয়। Array হলো একটি **ordered collection**, যেখানে প্রতিটি Element-এর অবস্থান বা Index থাকে। JavaScript-এ Index শুরু হয় **0** থেকে।

```javascript
const products = ['Mouse', 'Keyboard', 'Monitor'];
console.log(products[0]);          // Mouse
console.log(products[2]);          // Monitor
console.log(products.length);      // 3
console.log(products[3]);          // undefined
```

বাস্তব উদাহরণ: Shopping Cart-এর Items, API থেকে আসা Search Results, Students-এর Marks, Travel Packages-এর List, Booking Records—সব জায়গায় Arrays কাজে লাগে।

৬.২ Array তৈরি: Literal, Array.from, Array.of

```javascript
const empty = [];
const scores = [80, 95, 76];
const mixed = ['Dhaka', 2026, true, null];
const squares = Array.from({ length: 5 }, (_, index) => index ** 2);
const oneNumber = Array.of(5);  // [5]
const fiveSlots = Array(5);     // length 5, কিন্তু পাঁচটি initialized element নয়
console.log(squares);          // [0, 1, 4, 9, 16]
```

Array-তে বিভিন্ন Type রাখা সম্ভব হলেও একই ধরনের Data Structure ব্যবহার করলে Maintenance সহজ হয়। `Array(5)`-এর Sparse Slots এবং `[undefined, ...]` এক জিনিস নয়।

৬.৩ Index, Length এবং Boundary

```javascript
const cities = ['Dhaka', 'Chattogram', 'Sylhet'];
console.log(cities.at(-1));                 // Sylhet
console.log(cities[cities.length - 1]);   // Sylhet
cities[1] = 'Rajshahi';
console.log(cities); // ['Dhaka', 'Rajshahi', 'Sylhet']
```

`const` দিয়ে Array Declare করলেও তার Element পরিবর্তন করা যায়; `const` শুধু Variable-টিকে অন্য Array দিয়ে Reassign করতে দেয় না। `cities[100] = 'X'` করলে মাঝখানে অনেক Empty Slot তৈরি হয়—সাধারণ Data Model-এ এটি এড়িয়ে চলা ভালো।

৬.৪ Mutating বনাম Non-Mutating Methods

Method	কাজ	Original Array বদলায়?
`push`, `pop`	শেষে যোগ/সরানো	হ্যাঁ
`unshift`, `shift`	শুরুতে যোগ/সরানো	হ্যাঁ
`splice`	মাঝখানে যোগ/মুছা	হ্যাঁ
`sort`, `reverse`	সাজানো/উল্টানো	হ্যাঁ
`slice`	অংশ Copy	না
`map`, `filter`	নতুন Array তৈরি	না
`concat`	Array যুক্ত	না
`toSorted`, `toReversed`	Copy সাজানো/উল্টানো	না (আধুনিক Runtime)

```javascript
const cart = ['Pen', 'Book'];
cart.push('Bag');                 // original cart changes
const firstTwo = cart.slice(0, 2); // independent outer array
const updated = [...cart, 'Pencil']; // original untouched
console.log(cart, firstTwo, updated);
```

সতর্কতা: `splice(1, 1)`-এ দ্বিতীয় Argument হলো কতগুলো Element Delete হবে; শেষ Index নয়।

৬.৫ Array Traversal: for, for...of, forEach

```javascript
const prices = [100, 250, 75];
for (let index = 0; index < prices.length; index++) {
  console.log(index, prices[index]);
}
for (const price of prices) console.log(price);
prices.forEach((price, index) => console.log(index, price));
```

`forEach` নতুন Array Return করে না। `forEach`-এর Callback-এর মধ্যে `return` দিলে শুধু সেই Callback শেষ হয়, পুরো Iteration থামে না। Loop মাঝপথে থামাতে চাইলে `for...of` + `break` ব্যবহার করো।

৬.৬ map: প্রতিটি Element Transform করা

```javascript
const prices = [100, 200, 300];
const withVat = prices.map(price => price * 1.15);
console.log(withVat); // [114.99999999999999, 229.99999999999997, 345]
```

JavaScript Floating-Point Representation-এর কারণে Decimal Money Calculation-এ অদ্ভুত Fraction দেখা যেতে পারে। Monetary Calculation-এর জন্য Minor Units (যেমন পয়সা) ব্যবহার করা বেশি নিরাপদ।

```javascript
const priceInPaisa = [10000, 20000, 30000];
const afterTenPercent = priceInPaisa.map(paisa => Math.round(paisa * 1.10));
console.log(afterTenPercent); // [11000, 22000, 33000]
```

৬.৭ filter: শর্ত অনুযায়ী Data নির্বাচন

```javascript
const stock = [
  { name: 'Mouse', quantity: 12 },
  { name: 'Keyboard', quantity: 3 },
  { name: 'Monitor', quantity: 0 }
];
const lowStock = stock.filter(product => product.quantity <= 5);
console.log(lowStock.map(item => item.name)); // ['Keyboard', 'Monitor']
```

`filter` সব Matching Items দিয়ে নতুন Array দেয়। `find` প্রথম Matching Item দেয়, না পেলে `undefined`।

৬.৮ find, findIndex, some, every, includes

```javascript
const numbers = [4, 7, 10, 12];
console.log(numbers.find(n => n > 8));       // 10
console.log(numbers.findIndex(n => n > 8));  // 2
console.log(numbers.some(n => n % 2 !== 0)); // true
console.log(numbers.every(n => n > 0));      // true
console.log(numbers.includes(7));            // true
```

`some` অন্তত একটি Item শর্ত পূরণ করে কি না জানায়। `every` সব Item শর্ত পূরণ করে কি না জানায়। খালি Array-তে `every` true এবং `some` false Return করে—এটি Logic Design-এ মনে রাখতে হবে।

৬.৯ reduce: একটি Summary Value তৈরি

```javascript
const orders = [
  { price: 250, quantity: 2 },
  { price: 100, quantity: 3 }
];
const total = orders.reduce((sum, item) => sum + item.price * item.quantity, 0);
console.log(total); // 800
```

এখানে `0` হলো Initial Accumulator। Empty Array-তেও নিরাপদভাবে কাজ করতে Initial Value দেওয়া উচিত।

```javascript
const marks = [80, 95, 75, 90];
const sum = marks.reduce((total, mark) => total + mark, 0);
const average = marks.length ? sum / marks.length : 0;
console.log(average); // 85
```

`reduce` দিয়ে সব কিছু লেখা যায়, কিন্তু Readability কমে গেলে `map` / `filter` বা সাধারণ Loop বেশি উপযুক্ত।

৬.১০ Sorting: সবচেয়ে সাধারণ ভুল

```javascript
const prices = [9, 100, 20];
console.log([...prices].sort());             // [100, 20, 9] — lexicographic
console.log([...prices].sort((a, b) => a - b)); // [9, 20, 100]
console.log([...prices].sort((a, b) => b - a)); // [100, 20, 9]
```

`sort()` Original Array পরিবর্তন করে। Copy প্রয়োজন হলে `[...array].sort(...)` বা Supported Runtime-এ `toSorted(...)` ব্যবহার করো। Object Sorting-এ Numeric ও Text Field আলাদাভাবে Compare করতে হয়।

```javascript
const items = [
  { name: 'Pen', price: 25 },
  { name: 'Bag', price: 700 }
];
const byPrice = [...items].sort((a, b) => a.price - b.price);
const byName = [...items].sort((a, b) => a.name.localeCompare(b.name));
```

৬.১১ slice, splice এবং Immutable Updates

```javascript
const list = ['A', 'B', 'C', 'D'];
console.log(list.slice(1, 3)); // ['B', 'C']; original intact
const changed = [...list];
changed.splice(1, 2, 'X');
console.log(changed); // ['A', 'X', 'D']
console.log(list);    // ['A', 'B', 'C', 'D']
```

Immutable Update বলতে সাধারণত Existing Collection না বদলে নতুন Collection Return করা বোঝায়। UI State Management, Testing ও Debugging-এ এটি উপকারী।

৬.১২ Spread, Destructuring এবং Shallow Copy

```javascript
const [first, second, ...remaining] = ['A', 'B', 'C', 'D'];
console.log(first, second, remaining); // A B ['C', 'D']
const original = [{ name: 'Mouse', stock: 5 }];
const copy = [...original];
copy[0].stock = 2;
console.log(original[0].stock); // 2! nested object shared
```

Spread দিয়ে Array-এর **Shallow Copy** হয়। Nested Object একই Reference ধরে রাখে। একটি Object পরিবর্তন করতে হলে `{ ...item, stock: newStock }` দিয়ে সেই Object-ও Copy করো। গভীর Clone-এর জন্য Data Type ও Browser Support অনুযায়ী `structuredClone()` ব্যবহার করা যায়, তবে Function ও কিছু বিশেষ Object Clone করা যায় না।

৬.১৩ Multi-Dimensional Arrays

```javascript
const matrix = [
  [1, 2, 3],
  [4, 5, 6]
];
console.log(matrix[1][2]); // 6
const flattened = matrix.flat();
console.log(flattened); // [1, 2, 3, 4, 5, 6]
```

`flat()` Default-এ এক Level Flatten করে। Matrix, Timetable বা Grid Data-তে Nested Array দরকার হয়।

৬.১৪ Real-World Data Pipeline

ধরো Inventory Dashboard-এ User লিখল `mo`, Category বেছে নিল `Accessories`, তারপর Price অনুযায়ী Sort করল। পুরো Flow:

```
Raw Products Array
      ↓
Validate / Normalize
      ↓
Search + Category Filter
      ↓
Sort (without mutation)
      ↓
Render Table
      ↓
Summary Metrics (all products)
```

Search Results ও Overall Inventory Summary আলাদা রাখা জরুরি। Search-এ তিনটি Product দেখা গেলেও Overall Total Value যেন সব Product-এর হয়।

৬.১৫ Practical Project — Inventory Dashboard

Project Objective: Product Add, Stock Increase/Decrease, Delete, Search, Category Filter, Low-Stock Filter, Sort, Total Inventory Value এবং Browser Storage।

File Structure

```
source-code/chapter-06/
├── index.html
├── style.css
├── inventory.js
├── app.js
└── inventory.test.js
```

Run: `index.html` Browser-এ Open করো। External Package বা Build Step লাগে না। Automated Test-এর জন্য Node.js (18+) Install করে Project Directory-তে `node --test inventory.test.js` চালাও।

HTML-এর মূল অংশ

`index.html`-এ চারটি Summary Card, Product Form, Search/Filter Controls এবং একটি Table আছে। নিচের অংশে Product Entry Form-এর ধারণা দেখানো হলো:

```HTML
<form id="product-form">
  <label>ID <input name="id" required></label>
  <label>Name <input name="name" required></label>
  <label>Category <input name="category" required></label>
  <label>Price <input name="price" type="number" min="0" step="0.01" required></label>
  <label>Stock <input name="stock" type="number" min="0" step="1" required></label>
  <button>Add product</button>
</form>
```

Inventory Logic: Separation of Concerns

`inventory.js`-এ Data Operations রাখা হয়েছে; এটি DOM-এর উপর নির্ভরশীল নয়। `app.js`-এ Browser UI ও `localStorage` Handle করা হয়। ফলে Pure Functions আলাদাভাবে Test করা যায়।

```JAVASCRIPT
function addProduct(items, product) {
  const item = validateProduct(product);
  if (items.some(existing => existing.id === item.id)) {
    throw new Error('Duplicate product ID');
  }
  return [...items, item];
}
```

কেন নতুন Array? `push()` দিয়ে Original পরিবর্তন না করে `[...items, item]` Return করলে Function-এর Side Effect কমে।

Stock Update

```javascript
function updateStock(items, id, change) {
  if (!Number.isSafeInteger(change)) throw new Error('Invalid change');
  if (!items.some(item => item.id === id)) throw new Error('Not found');
  return items.map(item => {
    if (item.id !== id) return item;
    const stock = item.stock + change;
    if (stock < 0) throw new Error('Insufficient stock');
    return { ...item, stock };
  });
}
```

এই Design-এ কোনো Product-এর Stock Negative করা যায় না।

Filter + Sort

```javascript
const visible = Inventory.sortProducts(
  Inventory.filterProducts(products, {
    query: searchText,
    category: selectedCategory,
    lowStockOnly: true
  }),
  'price',
  'asc'
);
```

`filterProducts()` এবং `sortProducts()` দুটোই নতুন Array Return করে; Master Inventory পরিবর্তন হয় না।

Summary Calculation

```javascript
function summarize(items, threshold = 5) {
  return items.reduce((result, item) => {
    result.productCount++;
    result.totalUnits += item.stock;
    result.totalValueCents += Math.round(item.price * 100) * item.stock;
    if (item.stock <= threshold) result.lowStockCount++;
    return result;
  }, { productCount: 0, totalUnits: 0, totalValueCents: 0, lowStockCount: 0 });
}
```

Currency Integer Minor Units-এ হিসাব হচ্ছে। Dashboard-এ Display করার সময় ১০০ দিয়ে ভাগ করে BDT দেখানো হয়।

Safe DOM Rendering

User Input সরাসরি `innerHTML`-এ বসানো হয়নি। `textContent` ব্যবহার করা হয়েছে, যাতে Product Name-এ HTML-like Text থাকলেও তা Executable Markup না হয়।

```javascript
const cell = document.createElement('td');
cell.textContent = product.name;
row.append(cell);
```

Local Storage

```javascript
localStorage.setItem('inventory', JSON.stringify(products));
const restored = JSON.parse(localStorage.getItem('inventory') ?? '[]');
```

এটি Conceptual Example। প্রকৃত Project-এ Loaded Data পুনরায় Validate করা হয়েছে এবং Invalid/Corrupted Data Handle করার ব্যবস্থা আছে। Browser Storage Production Database বা Secure Server Storage-এর বিকল্প নয়।

Complete Runnable Files

বইয়ের Companion Source Code-এর `source-code/chapter-06/` Folder-এ **সম্পূর্ণ** `index.html`, `style.css`, `inventory.js`, `app.js` **এবং** `inventory.test.js` অন্তর্ভুক্ত।

৬.১৬ Debugging ও Common Mistakes

সমস্যা	কারণ	সমাধান
`sort()`-এ 100 আগে আসে	Default String Comparison	Numeric Comparator
`map()` Result `undefined`	Callback-এ `return` নেই	Explicit Return
Original Array বদলে গেছে	`sort`, `splice`, `push` Mutate করে	Copy First
`reduce` Empty Array-তে Error	Initial Value নেই	`reduce(callback, 0)`
Spread Copy-তে Nested Data বদলায়	Shallow Copy	Nested Object Copy
Price Calculation অদ্ভুত Decimal	Floating Point	Minor Units ব্যবহার
UI-তে Untrusted HTML	`innerHTML`	`textContent`

৬.১৭ Time Complexity

Operation	সাধারণ Complexity
Array Index Access	O(1)
`push`	Amortized O(1)

Operation	সাধারণ Complexity
`find`, `filter`, `map`	O(n)
`sort`	সাধারণভাবে O(n log n), Implementation-Dependent
`includes`	O(n)
`unshift`, `shift`	O(n)

এই Complexity Big-O-এর সাধারণ ধারণা; Engine Implementation ও Input Pattern অনুযায়ী বাস্তব Performance ভিন্ন হতে পারে।

৬.১৮ Exercises — নিজে চেষ্টা করো

1. `[5, 10, 15, 20]` থেকে শুধু ১০-এর বেশি সংখ্যাগুলো বের করো।
2. Product Prices `[100, 250, 400]`-এর ওপর ১০% Discount দিয়ে নতুন Array বানাও।
3. `['Dhaka', 'Sylhet', 'Dhaka']` থেকে Unique City বের করো।
4. Students Array থেকে সর্বোচ্চ Marks-এর Student খুঁজে বের করো। Empty Array Handle করো।
5. Stock Array-তে `quantity <= 5` হলে Low-Stock Product Count বের করো।
6. Numeric Array Descending Order-এ Sort করো, Original অপরিবর্তিত রেখে।
7. Array of Orders থেকে `price * quantity` যোগ করে Grand Total বের করো।
8. একটি Product-এর Stock Immutable পদ্ধতিতে Update করো।
9. `categorySummary()`-এর মতো Category অনুযায়ী Product Count বের করো।
10. Inventory Dashboard-এ `Out of Stock` Filter যোগ করো।

৬.১৯ Exercise Solutions

Solution 1

```javascript
console.log([5, 10, 15, 20].filter(n => n > 10)); // [15, 20]
```

Solution 2

```javascript
const prices = [100, 250, 400];
const discounted = prices.map(price => Math.round(price * 90) / 100);
console.log(discounted); // [90, 225, 360]
```

Solution 3

```javascript
const cities = ['Dhaka', 'Sylhet', 'Dhaka'];
console.log([...new Set(cities)]); // ['Dhaka', 'Sylhet']
```

Solution 4

```javascript
const students = [{ name: 'A', marks: 78 }, { name: 'B', marks: 91 }];
const topper = students.reduce((best, student) =>
  best === null || student.marks > best.marks ? student : best, null);
console.log(topper); // { name: 'B', marks: 91 }
```

Solution 5

```javascript
const inventory = [{ quantity: 8 }, { quantity: 3 }, { quantity: 0 }];
console.log(inventory.filter(item => item.quantity <= 5).length); // 2
```

Solution 6

```javascript
const nums = [3, 20, 1];
const descending = [...nums].sort((a, b) => b - a);
console.log(descending, nums); // [20, 3, 1] [3, 20, 1]
```

Solution 7

```javascript
const orders = [{ price: 100, quantity: 2 }, { price: 50, quantity: 3 }];
console.log(orders.reduce((sum, item) => sum + item.price * item.quantity, 0)); // 350
```

Solution 8

```javascript
const items = [{ id: 'A', stock: 5 }, { id: 'B', stock: 2 }];
const next = items.map(item => item.id === 'A' ? { ...item, stock: 7 } : item);
console.log(items[0].stock, next[0].stock); // 5 7
```

Solution 9

```javascript
const products = [{ category: 'Tech' }, { category: 'Books' }, { category: 'Tech' }];
const counts = products.reduce((acc, product) => {
  acc[product.category] = (acc[product.category] ?? 0) + 1;
  return acc;
}, Object.create(null));
console.log(counts.Tech, counts.Books); // 2 1
```

Solution 10

```javascript
const outOfStock = products => products.filter(item => item.stock === 0);
// Dashboard UI-তে নতুন Checkbox যোগ করে এই Filter ব্যবহার করো।
```

৬.২০ Knowledge Check — ১০টি MCQ

১. Array-এর প্রথম Element-এর Index কত? (A) 1 (B) 0 (C) -1 (D) 2

২. কোন Method নতুন Array Return করে? (A) push (B) pop (C) map (D) reverse

৩. `filter()` কী Return করে? (A) Boolean (B) প্রথম Match (C) Matching Elements-এর Array (D) Number

৪. `[10, 2].sort()`-এর Result কী? (A) `[2,10]` (B) `[10,2]` (C) Error (D) `undefined`

৫. `find()` Match না পেলে কী Return করে? (A) `null` (B) `[]` (C) `undefined` (D) `false`

৬. Original Array পরিবর্তন করে কোনটি? (A) slice (B) filter (C) map (D) splice

৭. `reduce()`-এর Initial Value কেন গুরুত্বপূর্ণ? (A) Styling (B) Empty Array Support (C) Sorting (D) String Conversion

৮. `[...items]` Nested Object-কে Deep Clone করে? (A) Yes (B) No

৯. `some()` কখন true? (A) সব Match (B) অন্তত একটি Match (C) কোনো Match নয় (D) Empty হলে সবসময়

১০. Monetary Total হিসাবের ভালো পদ্ধতি কোনটি? (A) String Concatenation (B) Minor Currency Units (C) Random Rounding (D) Array Join

Answer Key ও ব্যাখ্যা

প্রশ্ন	উত্তর	ব্যাখ্যা
1	B	Array Index ০ থেকে শুরু
2	C	`map()` প্রতিটি Element Transform করে নতুন Array দেয়
3	C	`filter()` সব Matching Element দেয়
4	B	Default Sort String Comparison করে
5	C	Match না পেলে `undefined`
6	D	`splice()` Original Array Mutate করে
7	B	Initial Value Empty Array Handle করে
8	B	Spread Shallow Copy করে
9	B	অন্তত একটি Match হলেই true
10	B	Integer Minor Units Floating-Point Error কমায়

৬.২১ Final Challenge

Inventory Dashboard-এ তিনটি Feature যোগ করো: (১) Category-wise Inventory Value, (২) JSON Export, (৩) Price Range Filter। প্রতিটি Feature-এর জন্য অন্তত একটি Automated Test লেখো। `localStorage` থেকে Restore করা Data-ও Validate করবে।

৬.২২ Chapter Summary

এই অধ্যায়ে Array Creation, Indexing, Iteration, Transformation, Filtering, Searching, Sorting, Reducing, Immutability, Nested Data এবং Complexity শিখলে। Practical Inventory Dashboard-এ এগুলো একত্রে ব্যবহার করা হয়েছে। পরের অধ্যায়ে **Objects, Destructuring ও Real-World Data Modeling** শেখা হবে, যেখানে Inventory-এর Product Object-এর গঠন আরও গভীরভাবে বোঝা যাবে।